MCP Tool Description Standards for AI Agent Integration

A technical specification for eWater API teams on structuring tool schemas for LLM-based agents

This document describes what eWater's MCP tool descriptions need to include so that AI agents (such as Tapha, the WhatsApp-based field assistant) can interpret eWater data correctly and respond clearly to field engineers — without surfacing raw binary payloads or misidentifying outgoing commands as incoming device data.

Prepared by
Tapha AI Integration Team

Date
July 2026

Version
1.0 — Initial

Classification
Technical / Confidential

1. Background & Context

1.1 What is an MCP Tool?

The **Model Context Protocol (MCP)** is an open standard that allows AI language models to call external data sources and APIs as structured "tools." Each tool has a *name*, a *description*, and an *input schema*. When a user asks the AI a question, the model decides which tool to call, sends it a structured argument, and receives a JSON result which it uses to form its reply.

1.2 How Tapha uses eWater MCP tools

Tapha is an AI field assistant deployed over WhatsApp. Field engineers can ask questions like "what is the status of asset 662?" and Tapha will call the appropriate eWater MCP tool, receive the result, and reply in plain language via WhatsApp.

Critically: **the LLM's only guide to understanding what a tool result means is the description text you provide.** The model cannot guess what `rawPayload` contains, what `Pipeline: CmdApi` signifies, or whether a "Dispense Limit" event is an error or a normal reading — unless those things are explained in the tool descriptions.

> ■ KEY PRINCIPLE
> Your tool descriptions are not documentation for human developers. They are **runtime instructions that a language model reads every single time it processes a user query.** They must be precise, complete, and written as if explaining the data to a smart but domain-naive reader.

1.3 What happens when descriptions are absent or minimal

Without adequate descriptions, the LLM makes its best guess — and gets it wrong in predictable ways:

- It includes raw binary payloads (**rawPayload**, hex byte strings, base64 blobs) verbatim in WhatsApp replies, making them unreadable.
- It confuses outgoing command logs with incoming device telemetry.
- It fails to apply protocol filters the user requests, because it does not know what the filter means.
- It misinterprets error-category events as normal status readings.
- It dumps 25-item technical record lists into a chat interface instead of summarising.

All of these failures were observed in production on 18 July 2026 and are documented in Section 3.

---

2. Description Requirements

2.1 Every tool description must answer these questions

For each tool, the `description` field must explain:

#	Question the description must answer	Why it matters
1	What does this tool return, in plain English?	The LLM must know what the result set represents before it can answer a question about it.

#	Question the description must answer	Why it matters
2	Which fields are human-readable and should be surfaced in replies?	Without this, the model includes every field — including binary garbage.
3	Which fields are internal/binary and should **never** appear in a reply?	The model needs an explicit instruction to omit binary fields — it will not infer this.
4	What do key enum/categorical values mean? (e.g. Pipeline, EventCategory)	Without semantics the model cannot interpret direction, severity, or state.
5	What does a zero or null value mean for critical fields?	Zero flow ticks in an Error event ≠ zero flow ticks in a Status event. Context changes meaning.
6	What is the right level of detail for a WhatsApp reply?	Instructs the model to summarise, not dump all fields into a chat message.

2.2 Required structure for each tool description

Tool descriptions should follow this structure inside the `description` string of the MCP schema:

```
[One-sentence purpose statement] Returns: [what the result set contains, field categories] Key fields: - fieldName: [plain-English meaning]. [Any enum values explained]. [What to do in a reply] - fieldName: [plain-English meaning]. OMIT FROM REPLIES — binary/internal data. Reply rules: - Summarise; never list more than 5 records in a WhatsApp reply - Never surface rawPayload, hexPayload, or any base64/hex field - If all records are outgoing commands (Pipeline=CmdApi), say so explicitly
```

2.3 Input parameters also need descriptions

When the tool takes parameters (e.g. `assetId`, `protocol`, `limit`), each parameter's `description` must explain valid values and what filtering does. The model uses this to decide whether and how to apply a filter.

> ■ **COMMON MISTAKE**
> Listing `protocol: { "type": "string" }` with no description means the LLM does not know what valid protocols are — so when a user says "ignore 4CCv1 entries" the model fetches everything unfiltered and attempts to post-process, often incorrectly.

3. Observed Failures — 18 July 2026

The following failures occurred in the "Tapha talk" group during active field use of the eWater MCP integration. They are direct consequences of missing or inadequate tool descriptions.

Failure 1 — Raw binary payloads in every reply

✗ **PROBLEM**

Every agent reply included full base64 and hex byte strings from the **rawPayload** field. These are unreadable in a WhatsApp message and consumed most of the response length.

User	Give me recent logs for asset id 662
Tapha ✗	**Log ID:** 1193838596836950018 **Raw Payload:** `44 00 00 10 01 0b 35 52 18 18 07 26 a7 35 00 00 d7 ff 49 e1 ff ff...` **Decoded:** Dispense Limit · Battery 12.6V · Flow Ticks: 0 · Flow Time: 257s *[Repeated for 25 entries]*

Root cause: No tool description instructs the model to omit rawPayload.

Required fix in tool description:

```
rawPayload: Raw binary device message (hex-encoded). Internal protocol data. NEVER include rawPayload in any reply — it is not meaningful to human readers. Always use the decoded fields instead.
```

Failure 2 — Outgoing commands returned as "last data received"

✗ PROBLEM
When asked "when was the last data received from asset 2910?", the agent returned a **CommandApi_1 / CmdApi** log — a command *sent to* the device, not data *received from* it. This gave completely wrong information about asset connectivity.

User	When was the last data received from the asset?
Tapha ✗	The last data received was July 18, 2026 at 16:28:26 Protocol: **CommandApi_1** · Pipeline: **CmdApi** Command Name: Read Tick Accumulator *(This is an outgoing command — not received data)*
Tapha ✓	Asset 2910 has not sent any telemetry recently. All recent log entries are outgoing commands (CmdApi). The device appears to be offline or not responding.

Required fix — pipeline field description:

```
pipeline: Communication direction. "CmdApi" — outgoing command sent FROM the platform TO the device. NOT data received from the device. "MQTT" / "UDP" — telemetry/response received FROM the device. When a user asks "when was data last received", only consider pipeline=MQTT or UDP. If only CmdApi records exist, state: "The device appears offline — only outgoing commands are visible, no telemetry has been received."
```

Failure 3 — Wrong event type returned for a named request

✗ PROBLEM
When asked for "a Get Status event", the agent returned a "Read Tick Accumulator" log — a completely different event — and presented it as if it matched the request.

Required fix — eventName field description:

```
eventName: The type of event this log entry represents. Examples: "Get Status", "Dispense Limit", "Health State", "Read Tick Accumulator", "Tap Top-Up", "Get Status reply". Use this field to filter results when the user asks for a specific event type. Do NOT return a different event type and present it as if it matches the request.
```

Failure 4 — Error events misread as neutral data

✗ PROBLEM

Flow ticks = 0 in a "Dispense Limit / Error" event was reported as "no water was dispensed" — a neutral observation. In fact, EventCategory=Error with event "Dispense Limit" means the device hit a limit and was forcibly cut off — an alert condition.

Required fix — eventCategory field description:

```
eventCategory: Severity and type classification. "Status" — normal health or state report. Treat as informational. "Error" — an anomalous or alert condition. Flag with ■ in replies. Do not describe Error event fields with neutral language. Specific error meanings: "Dispense Limit" (Error): Device reached its configured dispense threshold and cut off water flow. flowTicks=0 here means flow was stopped by the limit — not that no dispensing occurred during normal operation. "Health State" (Status): Routine status broadcast. Normal.
```

Failure 5 — Protocol filter not applied server-side

✗ PROBLEM

User asked to "ignore 4CCv1 protocols." The filtered response still included a 4CCv1 entry. The model fetched all logs and tried to exclude 4CCv1 by post-processing — incorrectly.

Required fix — excludeProtocols parameter description:

```
excludeProtocols (array of strings, optional): Protocols to exclude from the result set. Known values: "4CCv1", "Ewc2_5", "CommandApi_1", "Gadwall". Apply this filter at query time — do NOT fetch all records and filter client-side. Example: excludeProtocols=["4CCv1"] removes all 4CCv1 entries from results.
```

4. Field-by-Field Reference — Required Descriptions

The following table specifies the description text required for each key field across eWater MCP tools.

Field	In replies	Required description
`rawPayload` `hexPayload`	**OMIT**	Raw binary device message. Internal data. **Never include in any reply.** Use decoded fields only.
`pipeline`	**INTERPRET**	**CmdApi** = command sent TO device (outgoing). **MQTT/UDP** = telemetry FROM device (incoming). Only MQTT/UDP logs count as "data received."
`protocol`	**INTERPRET**	Device firmware protocol. Known: **Ewc2_5**, **4CCv1**, **CommandApi_1**, **Gadwall**. Use to filter by device type.
`eventCategory`	**FLAG**	**Status** = normal/informational. **Error** = anomalous — prefix reply with ■ and explain the operational implication.
`eventName`	**SURFACE**	Primary event type label. Filter by this when user requests a specific event. Never return wrong type.
`flowTicks`	**SURFACE**	Pulse count from flow meter. 0 in an Error event = flow cut off. 0 in a Status event = no flow in period.
`litres`	**SURFACE**	Volume dispensed in litres (computed from flowTicks × calibration). Primary volume indicator.
`batteryVoltage`	**SURFACE**	Battery voltage in volts. Normal: 11.5–13.0 V. Flag ■ if below 11.0 V.
`vsen1Adc vsen2Adc vsen3Adc`	**INTERPRET**	Analogue sensor ADC readings for auxiliary ports 1–3 (pressure, level, etc.). Surface raw value; note calibration needed for engineering units. 0 may mean no sensor connected.
`tamp1 / tamp2`	**FLAG**	Tamper detection. true = ■ Tamper Detected on input 1 or 2.
`lowBattery`	**FLAG**	true = ■ Battery Low. Flag prominently in replies.
`valveOn`	**SURFACE**	true = valve open (water flowing). false = valve closed. Surface as "Valve: open/closed."
`rfidDisabled`	**INTERPRET**	true = RFID reader disabled. Surface as "RFID: enabled/disabled."
`source`	**INTERPRET**	IMEI of the originating device. Surface as "Device IMEI" when asked to identify the source.
`logId`	**OMIT unless asked**	Internal database record ID. Do not include unless user explicitly requests log IDs.

Field	In replies	Required description
`correlationId` `userId (internal)`	**OMIT**	Internal platform tracking fields. Never surface in any reply.

5. Before / After — Tool Description Examples

5.1 Log query tool

✗ CURRENT (insufficient)

```
{ "name": "get_asset_logs", "description": "Get logs for an asset.", "parameters": { "assetId": { "type": "integer" }, "limit": { "type": "integer" } } }
```

✓ REQUIRED (complete)

```
{ "name": "get_asset_logs", "description": "Returns recent telemetry and command logs for a given asset. REPLY RULES: - Show max 5 records; summarise the rest. - NEVER include rawPayload/hexPayload. - Lead with: timestamp, eventName, eventCategory, 2-3 key decoded values. - Flag Error events with ■. - If all records are CmdApi, say device appears offline.", "parameters": { "assetId": { "type": "integer", "description": "Asset ID to query." }, "limit": { "type": "integer", "description": "Max records (default 25). Use small values for WhatsApp." }, "excludeProtocols": { "type": "array", "description": "Protocols to exclude. Known: 4CCv1, Ewc2_5, CommandApi_1, Gadwall. Apply server-side." } } }
```

5.2 Asset status tool

✗ CURRENT (insufficient)

```
{ "name": "get_asset_status", "description": "Returns the current status of an asset." }
```

✓ REQUIRED (complete)

```
{ "name": "get_asset_status", "description": "Returns latest status snapshot: connectivity, valve, battery, sensors, tamper flags. Key fields: - valveOn: true=flowing, false=closed. - batteryVoltage: normal 11.5-13V. - lowBattery: true = flag ■ Low Battery. - tamp1/tamp2: true = flag ■ Tamper. - pipeline: CmdApi=outgoing command, MQTT/UDP=device telemetry. - lastSeen: last MQTT/UDP log timestamp. OMIT: rawPayload, correlationId, userId, logId." }
```

6. Standard Reply Format for WhatsApp

This text should be included in **all** eWater tool descriptions. It tells the agent how to present data in a WhatsApp group chat context, regardless of which tool is called.

Boilerplate to include in every tool description

```
OUTPUT FORMAT FOR WHATSAPP: This tool result will be presented in a WhatsApp group to field engineers. 1. BREVITY: Show max 5 records per reply. If more exist, say: "Showing 5 of N — narrow your filter or ask for more." 2. BINARY FIELDS: Never include rawPayload, hexPayload, base64 strings, or hex byte sequences (e.g. "44 00 00 10 01 0b...") in any reply. 3. RECORD FORMAT: For each log, show only: [time] · [eventName] · [eventCategory] · [2-3 key values] Example: "18:54 · Health State · Status · Battery 12.6V · Valve closed" 4. ALERTS: Prefix Error-category events with ■. Example: "■ 18:57 · Dispense Limit · Error · Flow cut off after 257s" 5. OFFLINE DETECTION: If all recent records are pipeline=CmdApi, say: "Device appears offline — no telemetry received recently." 6. INTERPRETATION: Include operational meaning, not just raw values. Say "Battery: 12.6V (normal)" not just "12.6".
```

Example of a well-formed reply

User

Give me recent logs for asset 662

Tapha ✓

Asset 662 — last 5 events

■ **19:01** · Dispense Limit · Error · Flow stopped after 256s · Battery 12.6V (normal)
■ **18:57** · Dispense Limit · Error · Flow stopped after 257s · Usage count 18,913
18:54 · Health State · Status · Battery 12.6V · Valve closed · RFID disabled
■ **18:52** · Dispense Limit · Error · Flow stopped after 256s
18:22 · Get Status reply · Status · Pressure: No · Valve: open

Showing 5 of 38. Repeated Dispense Limit errors suggest the dispense limit may need reconfiguration.

---

7. Implementation Checklist

Use this checklist to verify each eWater MCP tool is ready for AI agent use:

#	Requirement	Priority
1	Tool description explains what the result set contains in plain English	**Required**
2	rawPayload / hexPayload explicitly flagged as OMIT	**Required**
3	pipeline field: CmdApi=outgoing vs MQTT/UDP=incoming explained	**Required**
4	eventCategory: Error vs Status semantics with alert guidance	**Required**

#	Requirement	Priority
5	eventName documented as a filterable discriminator	**Required**
6	All enum/categorical fields list valid values and meanings	**Required**
7	WhatsApp output format rules in every tool (brevity, no binary, alert prefixes)	**Required**
8	All filter parameters have descriptions listing valid values	**Required**
9	Internal fields (correlationId, userId, logId) flagged as OMIT	**Required**
10	vsen1–3 ADC fields explain physical meaning and units	**Recommended**
11	batteryVoltage normal range documented (11.5–13.0 V)	**Recommended**
12	flowTicks=0 context explained for Error vs Status events	**Recommended**
13	usageCounter documented as cumulative lifetime counter	**Recommended**